

Technischer Methodenbericht

Web Mining: Sentiment-Analyse von X (Twitter) und Aktienkursrenditen

Softwarearchitektur und Implementierungsdetails

Ilker

M.Sc. Data Science

5. Juli 2026

Inhaltsverzeichnis

1	Einleitung	2
2	Systemarchitektur	2
2.1	Verzeichnisstruktur	2
2.2	Datenfluss	2
2.3	Datenhaltung	3
3	Datenpipeline: X/Twitter-Crawler	3
3.1	Technologieauswahl: Playwright statt API	3
3.2	Datenbankschema	3
3.3	Such-Query-Design	4
3.4	Zweistufiges Ticker-Matching	4
3.5	Anti-Bot-Maßnahmen und Polling-Logik	4
3.6	Intraday-Preis-Snapshots	5
4	Datenpipeline: Historische Kurspreise	5
5	Sentiment-Analyse-Pipeline	5
5.1	Modell und Vorverarbeitung	5
5.2	Tägliche Aggregation und Follower-Gewichtung	6
5.3	Retweet-Separation	6
6	Korrelationsmodellierung	6
6.1	Daten-Merge und Overlap-Report	6
6.2	Methodische Implementierungsdetails	7
6.3	Ausgabeformate	8
7	Konfiguration, Tests und Reproduzierbarkeit	8
7.1	Konfigurationsmanagement	8
7.2	Automatisierte Tests	8
7.3	Notebook-Konventionen	8
8	Fazit	9
	Literatur	9

1 Einleitung

Dieser Bericht dokumentiert die softwaretechnische Umsetzung des WebMiningProject. Im Vordergrund stehen Architekturentscheidungen, Implementierungsdetails und das Zusammenspiel der Systemkomponenten. Der statistische Analyseteil wird nur insoweit beschrieben, als er konkrete Implementierungsentscheidungen begründet; inhaltliche Interpretation der Ergebnisse findet sich im Hauptbericht.

Das System gliedert sich in vier Hauptbereiche: (1) Datenerhebung via X (Twitter)-Crawler, (2) Kursdaten-Pipeline über Yahoo Finance, (3) Sentiment-Analyse mit einem vortrainierten Transformer-Modell sowie (4) statistische Korrelationsmodellierung in Jupyter-Notebooks. Die gesamte Implementierung erfolgte in Python 3.12.3 mit einer virtuellen Umgebung (.venv).

2 Systemarchitektur

2.1 Verzeichnisstruktur

Das Projekt folgt einer strikten Trennung zwischen Quellcode (src/), Analysenotebooks (notebooks/), Rohdaten (data/raw/) und verarbeiteten Daten (data/processed/):

Listing 1: Projektstruktur

```
1 WebMiningProject/
2 |-- src/
3 |   |-- scraping/      # X/Twitter-Crawler (Playwright)
4 |   |-- finance/       # Kursdaten-Pipeline (yfinance)
5 |   |-- sentiment/     # Placeholder (Implementierung in NB02)
6 |   '-- utils/         # Konfigurationshelfer
7 |-- notebooks/
8 |   |-- 01_data_quality_eda.ipynb    # Datenqualität & EDA
9 |   |-- 02_sentiment_analysis.ipynb  # Sentiment-Inferenz
10 |   '-- 03_correlation_modeling.ipynb # Korrelationsanalyse
11 |-- data/
12 |   |-- raw/prices/      # prices.csv + prices_meta.json
13 |   '-- processed/      # Parquet-Dateien (Sentiment, Korrelationen)
14 |-- configs/
15 |   |-- tickers.yml      # Fokus-Ticker & Start-Datum
16 |   '-- .env             # Secrets (gitignored)
17 '-- report/             # LaTeX-Quellen & Abbildungen
```

2.2 Datenfluss

Der Informationsfluss durch das System ist strikt unidirektional und in drei Stufen gegliedert (Abbildung 1):

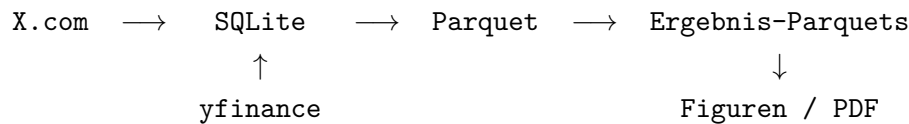


Abbildung 1: Datenfluss im WebMiningProject

2.3 Datenhaltung

Für persistente Speicherung werden zwei Formate eingesetzt:

- **SQLite** (`data/webmining.db`): Rohtweets, Engagement-Snapshots und Intraday-Preis-Snapshots. Betrieb im WAL-Modus mit aktivierten Foreign Keys.
- **Parquet / CSV**: Verarbeitete Daten für Notebooks. Parquet bietet typischere Spalten, schnelle Lesefade und effiziente Komprimierung gegenüber CSV bei gleichzeitiger Kompatibilität mit pandas und PyArrow.

3 Datenpipeline: X/Twitter-Crawler

3.1 Technologieauswahl: Playwright statt API

X (Twitter) hat im Jahr 2023 den kostenlosen API-Zugang für Dritttaggregation weitgehend eingestellt. Als Alternative wurde ein browserbasierter Crawler auf Basis von **Playwright** (headless Chromium) implementiert. Playwright erlaubt die Injektion echter Session-Cookies und damit die Nutzung des regulären X-Frontends ohne kostenpflichtige API-Lizenz.

Der Crawler ist explizit für den Betrieb auf einem **Raspberry Pi 4** (ARM64) ausgelegt. Da Playwright keine vorgebauten ARM-Chromium-Binaries bereitstellt, verweist die Konfiguration auf den Systemchromium (`chromium-browser`), der via `apt` installiert wird. Das Skript `setup_pi.sh` automatisiert die vollständige Einrichtung inklusive Cron-Jobs (Discovery Mo–Fr um 09:00, 12:00, 15:00 Uhr; Polling alle 30 Minuten).

3.2 Datenbankschema

Die SQLite-Datenbank enthält drei Tabellen:

Tabelle 1: SQLite-Schema (`data/webmining.db`)

Tabelle	PK	Wesentliche Spalten
tweets	tweet_id (TEXT)	author, author_followers, content, posted_at (ISO 8601), symbol, likes, retweets
tweet_snapshots	id (INT)	tweet_id (FK), crawled_at (UTC), likes, retweets
price_snapshots	id (INT)	tweet_id (FK), symbol, timestamp (1 min OHLCV), price, volume

`store_tweet()` nutzt den `UNIQUE-Constraint` auf `tweet_id` zur idempotenten Ausführung: ein `IntegrityError` wird abgefangen und als `False` zurückgegeben (Duplikat), während eine erfolgreiche Insertion `True` zurückgibt. Dies ermöglicht restart-sichere Crawler-Läufe.

3.3 Such-Query-Design

Die Konfiguration (`x_config.py`) definiert über 100 Suchanfragen in sechs Sprachen (EN, DE, FR, IT, JA, SV). Jede Query kombiniert mehrere Signaltypen:

Listing 2: Beispiel-Queries (`x_config.py`)

```
1 QUERIES = [  
2     "$TSLA OR #Tesla OR #TeslaStock",           # Cashtag + Hashtag EN  
3     "#Tesla #Aktie OR #TeslaAktie",             # Hashtag DE  
4     "Tesla OR $TSLA lang:ja",                   # Japanisch  
5     "$BYD OR #BYDCar OR #\u6bd4\u4e9a\u8fea",     # BYD + CJK -  
        Zeichen  
6     "$NIBE OR #NIBEHeatPump OR #Waermepumpe",   # NIBE + Sektorterm  
7 ]
```

Server-seitig werden `min_faves:` und `since:` in die URL injiziert. Client-seitig filtert der Crawler nach `MIN_LIKES = 100` und `MIN_FOLLOWERS = 100`.

3.4 Zweistufiges Ticker-Matching

Da Tweets keinen einheitlichen Ticker-Tag tragen, erfolgt die Zuordnung zu Fokus-Unternehmen in zwei Passes:

1. **Pass 1 (direkt):** Cashtag-Aliases werden bekannten Symbolen zugeordnet (z. B. `$BYD` $\rightarrow$ `BYDDY`, `$VOW` $\rightarrow$ `VWAGY`).
2. **Pass 2 (Regex):** Tweets ohne Symbolzuordnung werden anhand regulärer Ausdrücke auf Firmennamen und Marken geprüft. Wortgrenzen wurden auf `(?<!\w)` angepasst, um zusammengesetzte Hashtags (`#TeslaStock`, `#NIBEHeatPump`) zu erfassen.

3.5 Anti-Bot-Maßnahmen und Polling-Logik

Zur Vermeidung von Sperrungen implementiert der Crawler mehrere Strategien:

- **Zufällige Delays:** 3–6 s zwischen Requests (`REQUEST_DELAY_MIN/MAX`).
- **Lazy Follower-Abfrage:** Follower-Zahlen werden nur für Tweets erhoben, die den Like-Filter bestanden haben (Profilbesuche erhöhen das Entdeckungsrisiko).
- **Adaptiver Velocity-Filter:** Nach mindestens 4 Snapshots (`POLL_MIN_SNAPSHOTS`) wird ein Tweet nur weiter verfolgt, wenn seine Engagement-Velocity $\geq 0,3$ Likes/Minute beträgt. Dieser Filter reduziert nutzlose Polling-Zyklen für stagnante Tweets erheblich und wird über eine korrelierte SQL-Subquery effizient berechnet.

Das Polling-Fenster beträgt 8 Stunden (alle 30 Minuten), was bis zu 16 Snapshots je Tweet und damit eine verwertbare Engagement-Zeitreihe erzeugt.

3.6 Intraday-Preis-Snapshots

Für jeden neu gespeicherten Tweet lädt der Crawler 1-Minuten-OHLCV-Daten im Fenster $[t_{\text{Tweet}} - 30 \text{ min}, t_{\text{Tweet}} + 120 \text{ min}]$ herunter und speichert sie in `price_snapshots`. Geschlossene Märkte (Wochenenden, außerhalb NYSE-Öffnungszeit 09:30–16:00 ET) werden vorab erkannt und übersprungen.

4 Datenpipeline: Historische Kurspreise

Der Kursdaten-Fetcher (`src/finance/price_fetcher.py`) lädt historische OHLCV-Tagesdaten für alle Fokus-Ticker über `yfinance`. Das Design folgt dem Prinzip einer wachsenden, inkrementell aktualisierten CSV-Datei:

Listing 3: Inkrementelles Laden (`price_fetcher.py`)

```
1 # Pro Ticker nur fehlende Tage nachladen
2 for ticker in tickers:
3     last = existing[existing["Ticker"] == ticker]["Date"].max()
4     start = last + timedelta(days=1)
5     new_data = yf.download(ticker, start=start, end=end,
6                           auto_adjust=True, progress=False)
```

Neben der CSV wird eine JSON-Metadatendatei (`prices_meta.json`) mit Zeitstempel, Ticker-Liste, Zeilenanzahl, Datumsbereich und `yfinance`-Version gespeichert. Ein 1,5-Sekunden-Delay zwischen Ticker-Downloads verhindert Rate-Limiting bei Yahoo Finance.

Für die Analyse wird ausschließlich `Adj Close` (split- und dividendenbereinigt) als Kursreferenz verwendet. Kursausreißer wurden in NB01 über die $\text{IQR} \times 3$ -Regel je Ticker identifiziert; fehlende Handelstage wurden mit der Bibliothek `exchange_calendars` (NYSE-Kalender) überprüft.

5 Sentiment-Analyse-Pipeline

5.1 Modell und Vorverarbeitung

Die Sentiment-Inferenz erfolgt in NB02 mit dem vortrainierten Modell `cardiffnlp/twitter-xlm-roberta-base` (Loureiro u. a. 2022) über die Hugging Face Transformers-Bibliothek. Die Wahl dieses Modells begründet sich in seiner Mehrsprachigkeit: Der Datensatz enthält Tweets auf Englisch, Deutsch, Französisch und Japanisch, die ohne Übersetzungsumweg verarbeitet werden können.

Vor der Inferenz werden Texte bereinigt:

Listing 4: Textbereinigung vor Modell-Inferenz

```
1 import re
2
3 def clean_text(text: str) -> str:
4     text = re.sub(r"https?://\S+", "", text) # URLs entfernen
5     text = re.sub(r"[$@]\w+", "", text) # Cashtags & Mentions
```

```

6     return text.strip()
7
8 # Nur Tweets mit mind. 20 verbleibenden Zeichen inferieren
9 df["has_content"] = df["content"].map(clean_text).str.len() >= 20

```

Das Modell liefert Wahrscheinlichkeiten für drei Klassen; der kontinuierliche Compound-Score ist:

$$\text{xlm_compound} = P(\text{positive}) - P(\text{negative}) \in [-1, 1] \quad (1)$$

Die Inferenz läuft im Batch-Modus (BATCH_SIZE=32) mit optionaler GPU-Beschleunigung. Ein Caching-Mechanismus (FORCE_RERUN=False) verhindert redundante GPU-Nutzung bei unveränderten Eingabedaten.

5.2 Tägliche Aggregation und Follower-Gewichtung

Tweet-Level-Scores werden je Fokus-Ticker und Kalendertag aggregiert. Das primäre Analyse-signal ist der follower-gewichtete Compound-Score:

$$\text{weighted_xlm} = \frac{\sum_i f_i \cdot c_i}{\sum_i f_i} \quad (2)$$

wobei f_i die Follower-Zahl und c_i der Compound-Score von Tweet i . Tage, an denen alle Autoren null Follower haben, erhalten NaN (werden beim Inner-Join in NB03 implizit ausgeschlossen). Die Ausgabedatei `daily_sentiment_original.parquet` enthält 508 Zeilen über 12 Ticker.

5.3 Retweet-Separation

Retweets (erkennbar am Präfix RRT @) werden in separate Parquet-Dateien ausgelagert (`*_retweets.parquet`), um das Primärsignal nicht durch bloße Multiplikation originärer Meinungen zu verzerren. Im vorliegenden Datensatz enthielten alle gecrawlten Inhalte originären Text; die Separation ist methodisch korrekt implementiert und für Folgestudien wiederverwendbar.

6 Korrelationsmodellierung

6.1 Daten-Merge und Overlap-Report

NB03 führt Sentiment-Daten und Kurspreise über einen Inner-Join auf (`focus_ticker`, `date`) zusammen. Statt Ticker mit zu wenig Daten stillschweigend auszuschließen, erzeugt das Notebook einen **Overlap-Report** mit methoden-spezifischen Eligibility-Flags, der als `overlap_report.parquet` persistent gespeichert wird:

Tabelle 2: Eligibility-Schwellwerte im Overlap-Report

Methode	Mindestvoraussetzung
Pearson/Spearman	≥ 5 gepaarte Beobachtungen je Lag
Rolling-Korrelation	≥ 30 überlappende Tage
Granger-Test	≥ 30 Beobachtungen
Event-Study	≥ 20 Basistage <i>und</i> ≥ 5 Ereignisse je Richtung

6.2 Methodische Implementierungsdetails

Multiple Testing (BH-FDR). Da Pearson/Spearman-Tests für bis zu 10 Ticker $\times$ 4 Lags = 40 simultane Hypothesen durchgeführt werden, korrigiert das Notebook alle p -Werte mit der Benjamini-Hochberg-FDR-Methode:

Listing 5: BH-Korrektur via statsmodels

```
1 from statsmodels.stats.multitest import multipletests
2
3 _, p_adj, _, _ = multipletests(corr_df["pearson_p"].values, method="
    fdr_bh")
4 corr_df["pearson_p_adj"] = p_adj
5 corr_df["pearson_significant"] = p_adj < 0.05
```

Konfidenzintervalle (Fisher-z). Zu jedem Pearson- r wird ein 95 %-KI über die Fisher- z -Transformation berechnet ($SE_z = (n - 3)^{-1/2}$), das die Schätzunsicherheit bei kleinen Stichproben transparent macht.

Stationaritätsprüfung (ADF). Vor den Granger-Tests werden alle Eingangszeitreihen mit `adfuller()` aus `statsmodels` auf Einheitswurzeln geprüft. Alle vier getesteten Ticker waren auf beiden Reihen stationär ($p < 0,001$) – die Voraussetzung war erfüllt.

Event-Study-Benchmark. Als Markt-Benchmark für abnormale Renditen wird der S&P 500 (`^GSPC`) via `yfinance` heruntergeladen, anstatt den internen Focus-Ticker-Mittelwert zu verwenden, der zu einem zirkulären Vergleich geführt hätte:

Listing 6: Externer S&P-500-Benchmark

```
1 spx_raw = yf.download("^GSPC", start=spx_start, end=spx_end,
2                        auto_adjust=True, progress=False)
3 market_ret = spx_raw["Close"].pct_change().rename("market_return")
4 merged2["abnormal_return"] = merged2["return"] - merged2["
    market_return"]
```


6.3 Ausgabeformate

Tabelle 3: Ergebnis-Parquets aus NB03 (data/processed/)

Datei	Inhalt
correlation_results.parquet	r , KI, p_{roh} , p_{adj} , n je (Ticker, Lag)
granger_results.parquet	p -Werte (F-Test) je (Ticker, Richtung, Lag)
event_study_results.parquet	Mittlere abnormale Rendite je (Ticker, Richtung, t)
overlap_report.parquet	Coverage und Eligibility-Flags je Fokus-Ticker

7 Konfiguration, Tests und Reproduzierbarkeit

7.1 Konfigurationsmanagement

Secrets (X-Session-Cookies) werden ausschließlich über `configs/.env` (gitignored) bereitgestellt; eine kommentierte Vorlagendatei `configs/.env.example` ist versioniert und dokumentiert alle erwarteten Variablen. Analyse-Parameter sind zentral definiert:

- `configs/tickers.yml`: 13 Fokus-Ticker + `start_date`
- `src/scraping/x_config.py`: alle Crawler-Parameter als benannte Konstanten (`MIN_LIKES`, `POLL_INTERVAL_MINUTES`, ...)

Jedes Notebook definiert Schwellwerte am Anfang explizit, sodass Reproduzierbarkeit ohne Kenntnis externer Konfigurationsdateien gegeben ist.

7.2 Automatisierte Tests

Tabelle 4: pytest-Testdateien in `tests/`

Datei	Geprüfte Komponente
<code>test_db.py</code>	SQLite-Schema, <code>store_tweet()</code> , Duplikaterkennung
<code>test_price_fetcher.py</code>	Inkrementelles Laden, Merge-Logik, Metadaten-JSON
<code>test_x_crawler.py</code>	Parser-Funktionen (<code>_parse_stat()</code> , Filterlogik)

7.3 Notebook-Konventionen

- Outputs werden vor Commits geleert (*Clear All Outputs*).
- Zufallsprozesse nutzen feste Seeds (`seed=42`).
- Figuren werden via `_save_fig()` automatisch nach `report/figures/` exportiert (`FIG_DPI=300`).
- Parquet-Artefakte am Ende jedes Notebooks gespeichert; Folgenotebooks lesen anschließend diese Dateien (keine direkten SQLite-Abhängigkeiten in NB02/NB03).

8 Fazit

Die implementierte Pipeline deckt den vollständigen Datenlebenszyklus von der Erhebung bis zur statistischen Auswertung ab. Besonderes Augenmerk wurde auf drei Aspekte gelegt:

1. **Modularität:** Klare Trennung zwischen Scraping, Preisdaten und Analyse; jede Komponente ist unabhängig testbar und austauschbar.
2. **Reproduzierbarkeit:** Parquet-Artefakte als unveränderliche Zwischenstände, explizite Parameter, versionierte Konfigurationsvorlagen.
3. **Methodische Korrektheit:** BH-FDR-Korrektur für Multiple Testing, ADF-Stationaritätstests vor Granger-Kausalitätstests, externer S&P 500-Benchmark für die Event-Study.

Erweiterungspotenzial besteht insbesondere in drei Bereichen: (1) Intraday-Analyse auf Basis der bereits gespeicherten `price_snapshots`-Daten, (2) Log>Returns für normalverteilungsnahere Residuenverteilungen im Granger-Modell sowie (3) Kreuzvalidierung zur Reduktion des Data-Snooping-Bias bei der Lag-Selektion.

Literatur

Loureiro, Daniel u. a. (2022). „TimeLMs: Diachronic Language Models from Twitter“. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, S. 251–260. DOI: [10.18653/v1/2022.acl-demo.25](https://doi.org/10.18653/v1/2022.acl-demo.25).